

Session 3: Shell Programming

1. Decision Loops in Shell Programming

Shell scripts use decision-making statements and loops to control the flow of execution based on conditions.

1.1 if-else Statement

- Used to execute a block of code based on a condition.

- **Syntax:**

```
if [ condition ]
then
    commands
else
    commands
fi
```

- **Example:**

```
read -p "Enter a number: " num
if [ $num -gt 10 ]
then
    echo "The number is greater than 10."
else
    echo "The number is less than or equal to 10."
fi
```

1.2 test Command

- Used for evaluating conditions within decision-making.

- **Examples:**

- `[ -f file.txt ]`: Checks if `file.txt` exists and is a regular file.
 - `[ $num -eq 10 ]`: Checks if `$num` is equal to 10.
-

1.3 Nested if-else Statement

- An if statement within another if block.

- **Example:**

```
read -p "Enter a number: " num
if [ $num -gt 0 ]
then
    if [ $num -lt 100 ]
    then
        echo "The number is between 0 and 100."
    else
        echo "The number is greater than or equal to 100."
    fi
else
    echo "The number is less than or equal to 0."
```

1.4 case Statement

- Simplifies complex decision-making with multiple conditions.
- **Syntax:**

```
case variable in
    pattern1)
        commands ;;
    pattern2)
        commands ;;
    *)
        default_commands ;;
esac
```

- **Example:**

```
read -p "Enter a day: " day
case $day in
    "Monday")
        echo "Start of the work week." ;;
    "Friday")
        echo "Almost the weekend!" ;;
    *)
        echo "It's just another day." ;;
esac
```

2. Loops in Shell

2.1 while Loop

- Repeats commands as long as a condition is true.
- **Syntax:**

```
while [ condition ]
do
    commands
done
```

- **Example:**

```
count=1
while [ $count -le 5 ]
do
    echo "Count: $count"
    count=$((count + 1))
done
```

2.2 until Loop

- Similar to `while` but runs until the condition becomes true.
- **Syntax:**

```
until [ condition ]
do
    commands
done
```

- **Example:**

```
count=1
until [ $count -gt 5 ]
do
    echo "Count: $count"
    count=$((count + 1))
done
```

2.3 for Loop

- Iterates through a sequence of values.
- **Syntax:**

```
for variable in list
do
    commands
done
```

- **Example:**

```
for i in 1 2 3 4 5
do
    echo "Number: $i"
done
```

- **Range Example:**

```
for i in {1..5}
do
    echo "Number: $i"
done
```

3. Regular Expressions in Shell

- Used for pattern matching in text processing commands like **grep**, **sed**, and **awk**.
-

Basic Regular Expressions (Regex) Syntax

- **.**: Matches any single character.
 - Example: `grep "c.t" file.txt` (matches "cat", "cut").
- *****: Matches zero or more occurrences of the preceding character.
 - Example: `grep "ho*" file.txt` (matches "h", "ho", "hoo").
- **[]**: Matches any character inside the brackets.
 - Example: `grep "[abc]" file.txt` (matches "a", "b", or "c").
- **^**: Matches the beginning of a line.
 - Example: `grep "^Start" file.txt` (matches lines starting with "Start").

- `$`: Matches the end of a line.
 - Example: `grep "end$" file.txt` (matches lines ending with "end").
-

4. Arithmetic Expressions

Shell supports basic arithmetic operations using `$( (expression) )`.

- **Examples:**

```
a=5
b=3
sum=$((a + b))
echo "Sum: $sum"
diff=$((a - b))
echo "Difference: $diff"
prod=$((a * b))
echo "Product: $prod"
div=$((a / b))
echo "Division: $div"
```

5. More Examples in Shell Programming

Example 1: Check if a File Exists

```
read -p "Enter the file name: " file
if [ -f $file ]
then
    echo "File exists."
else
    echo "File does not exist."
fi
```

Example 2: Print a Multiplication Table

```
read -p "Enter a number: " num
for i in {1..10}
do
    echo "$num x $i = $((num * i))"
done
```

Example 3: Counting Words in a File

```
read -p "Enter file name: " file
if [ -f $file ]
then
    word_count=$(wc -w < $file)
    echo "The file has $word_count words."
else
    echo "File not found!"
fi
```

This detailed explanation covers all aspects of shell programming decision loops, regular expressions, arithmetic expressions, and practical examples, ensuring you can understand and apply these concepts effectively.